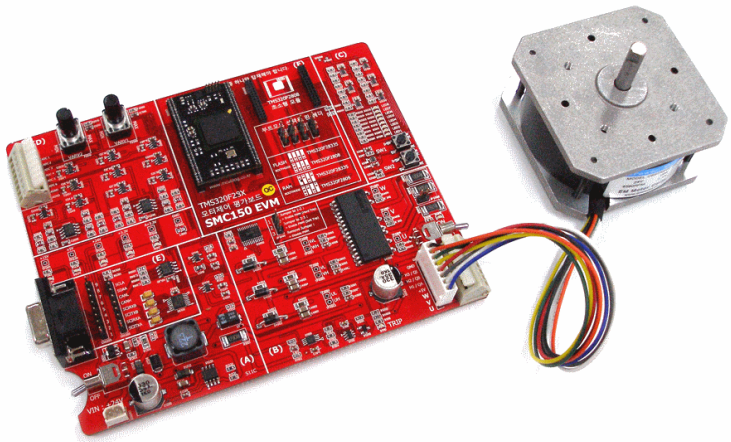
* 본 자료는 모터구동의 한 예제로만 사용하십시오. 어플리케이션 적용으로 인한 문제에 대해서는 싱크웍스에서 책임지지 않습니다. 또한, 기술적인 지원도 할 수 없음을 알려드립니다.

* 본 예제 프로그램은 Texas Instruments(TI)사에서 TMS320F28X 프로세서 사용자들을 대상으로 공개한 Digital Motor Control Library를 기초로 만들어졌으며, SMC150 개발보드에 적합하도록 일부 수정되었습니다. (Digital Motor Control Library 최신버전은 TI의 ControlSUITE를 통해 받아보실 수 있습니다.)

*Revision 2012.07.30 (v1.5)

Hall-Effect Sensor를 이용한 3상 BLDCM의 속력제어



*Revision History

날 짜	내 용
12. 01. 11.	- V1.0 / 초판
12. 07. 30.	- V1.5 / CCS V5용 예제 프로젝트 추가, TMS320F28069용 예제 추가

목차

1 SMC150 개발보드 환경설정 4

1.1 SMC150 개발보드에 BLDCM 연결하기 5
1.2 BLDCM 사양 및 정격/성능 (필름코일 타입) 6
1.3 전원 공급 예 7
1.4 PWM 출력 및 TRIP 제어용 점퍼 핀-헤더 설정 (JP3000) 8

2 예제 실습 10

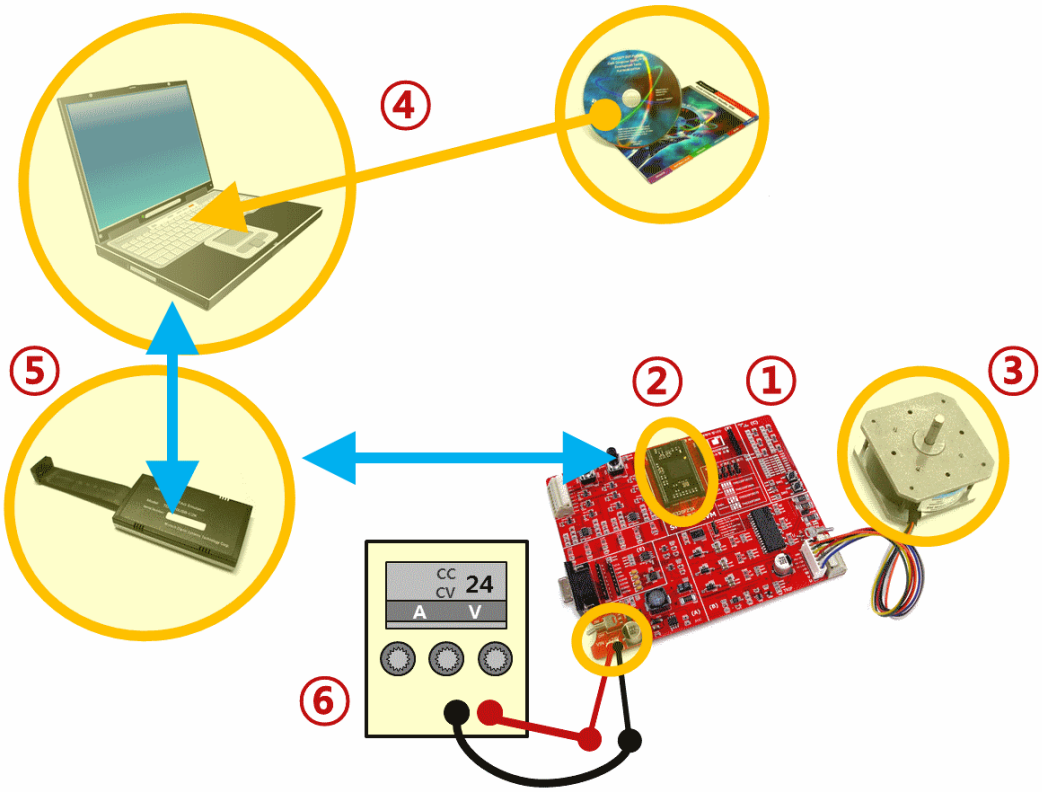
2.1 예제 디렉터리 색인 10
2.2 예제 구성 12
2.3 실습 따라 하기 (BUILD LEVEL 1) 13
2.4 실습 따라 하기 (BUILD LEVEL 2) 15

3 실습 끝내기 절차 18

1 SMC150 개발보드 환경설정

예제 프로그램을 실행하기 위해 아래의 개발환경들이 필요합니다.

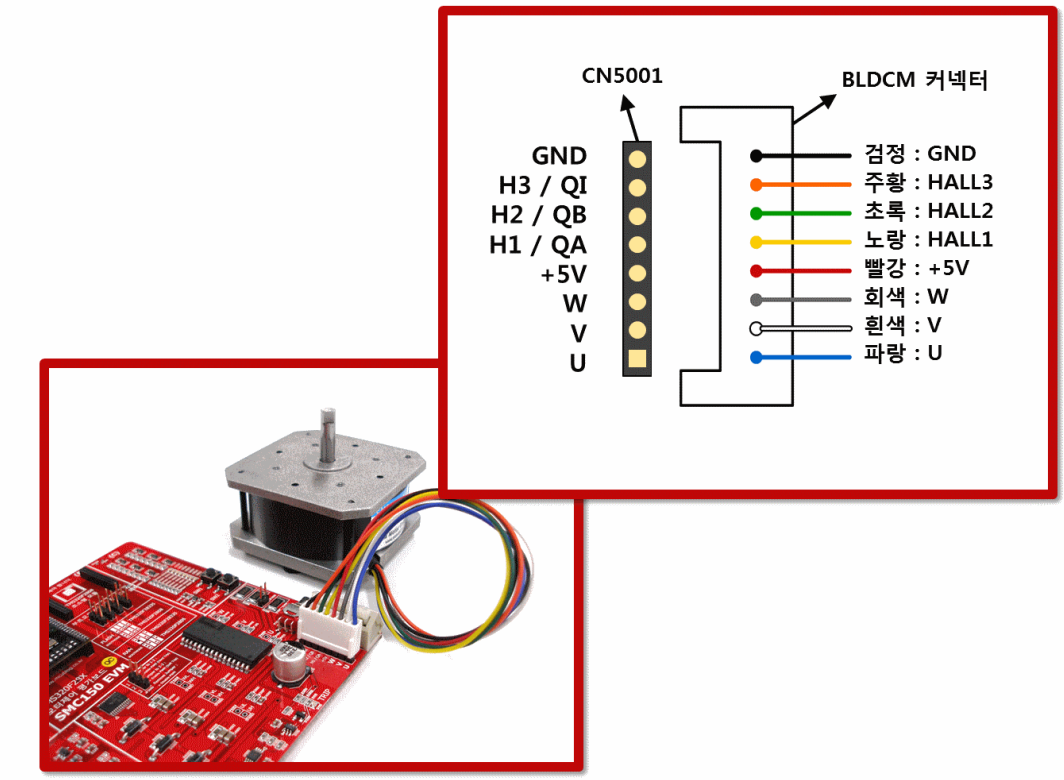
① **SMC150 개발보드** (3상 모터제어 개발보드)
② **프로세서 모듈** (TMS320F28335 초소형, TMS320F2808 초소형)
③ **15W 3Phase BLDCM** (with Hall-Effect Sensor)
④ **Code Composer Studio**(v3.3 혹은 그 이상의 버전)가 설치된 IBM 호환 PC
⑤ **JTAG 에뮬레이터** (TDS510USB-C2K 혹은 동종의 JTAG 에뮬레이터)
⑥ **DC 전원공급기(DC Power Supply) 1개** (DC +24V 공급이 가능한 제품)



[그림 1-1] – 예제 프로그램 실행을 위한 개발환경

1.1 SMC150 개발보드에 BLDCM 연결하기

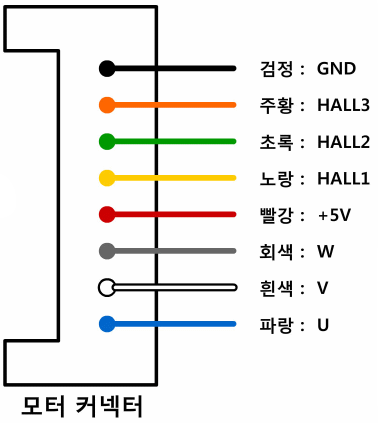
SMC150 개발보드와 함께 판매되는 Brushless DC Motor는 15W급의 3상 제품이며, Hall-Effect Sensor를 내장하고 있습니다. 이 BLDCM은 아래의 그림 1-2와 같이 SMC150 개발보드의 **CN5001 핀-헤더**에 연결할 수 있습니다.



[그림 1-2] – SMC150 개발보드와 BLDCM의 연결

1.2 BLDCM 사양 및 정격/성능 (필름코일 타입)

SMC150 개발보드와 함께 판매되는 3상 Brushless DC Motor는 Hall-Effect Sensor를 내장하고 있으며, 각 신호 및 전원과 Lead-wire 색의 관계는 아래의 그림 1-3과 같습니다.

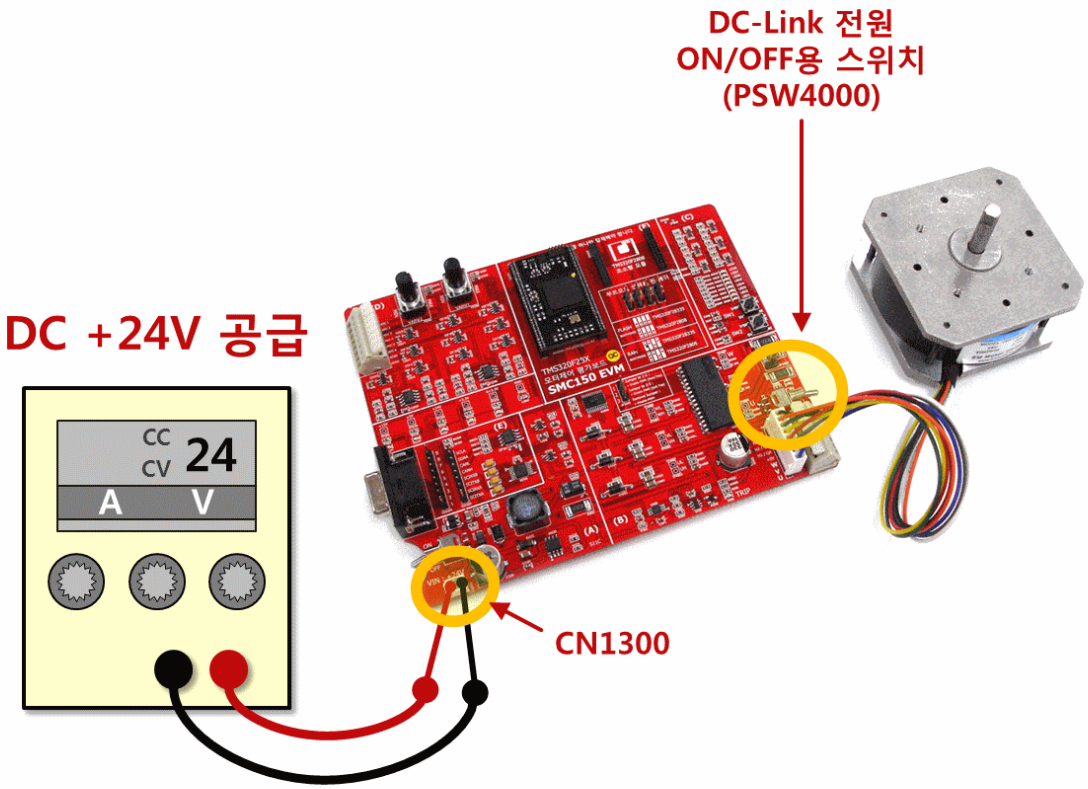


[그림 1-3] – BLDCM 커넥터 정의

구분	사양
Rated Voltage	DC 24V
Rated Current	1.1A
Rated Power	15W
Rated Speed	4,580RPM
Rated Torque	0.03N.m
Max. Output Power	28W
Max. Efficiency	58%
No Load Speed	5,400RPM
Stall Torque	0.21N.m
Terminal Resistance	4.5Ω
Shaft Diameter	Φ 6
Bearing Type	Ball
Weight	423g
Operating Temperature Range	-20℃ ~ +80℃

[표 1-1] – SMC150 개발보드와 함께 판매되는 BLDCM의 사양 및 정격

1.3 전원 공급 예



[그림 1-4] – SMC150 개발보드의 전원 공급 예

주의: 전력 변환이나 모터 구동과 같은 어플리케이션은 고-전압 혹은 고-전류에 노출되어 있기 쉽습니다. 이는 부주의로 인한 사고 발생시 장비들을 손상시킬 뿐만 아니라 여러분에게도 위험을 줄 수 있습니다. 따라서 실습을 진행함에 있어 명시된 절차를 필히 지켜주셔야 합니다. 또한 전류 수치의 확인 및 제한이 용이한 DC 전원공급기의 사용을 권고 드리며, 안전한 실습을 위해, 예제 프로그램의 실행을 멈추기 전, 반드시 DC-Link에 공급된 전원을 차단해주십시오. (PSW4000 스위치 사용)

1.4 PWM 출력 및 Trip 제어용 점퍼 핀-헤더 설정 (JP3000)

SMC150 개발보드에 탑재된 TMS320F28X 프로세서의 3.3V Level PWM 출력 신호들은 Voltage Translator(SN74LVC4245A, U3000)를 거쳐, 5.0V Level로 가공되고, 모터 드라이버 IC(Smart Power Module, U4000)로 전달됩니다. 여기에 사용된 Voltage Translator는 /OE 핀을 가지고 있어, 해당 핀의 제어에 따라, 모터 드라이버 IC로 PWM을 전달(Enable)하거나 혹은 차단(Disable)할 수 있습니다. 또한, SMC150 개발보드에는 Texas Instruments사의 INA200(High-Side Measurement Current-Shunt Monitor with Open-Drain Comparator and Reference) 소자를 이용한 과-전류 Trip 회로가 설계되어 있습니다. 이 회로는 DC-Link에 흐르는 전류가 설정된 수치(1A)를 초과할 경우, Trip 펄스를 발생시키게 됩니다. 이렇게 발생된 Trip 펄스는 TMS320F28X 프로세서에 전달되며, 동시에 JP3000 점퍼 핀-헤더(2.54mm 1x3)를 거쳐서, PWM 출력을 모터 드라이버 IC로 전달하는 Voltage Translator의 /OE 핀에도 전달되게 됩니다.

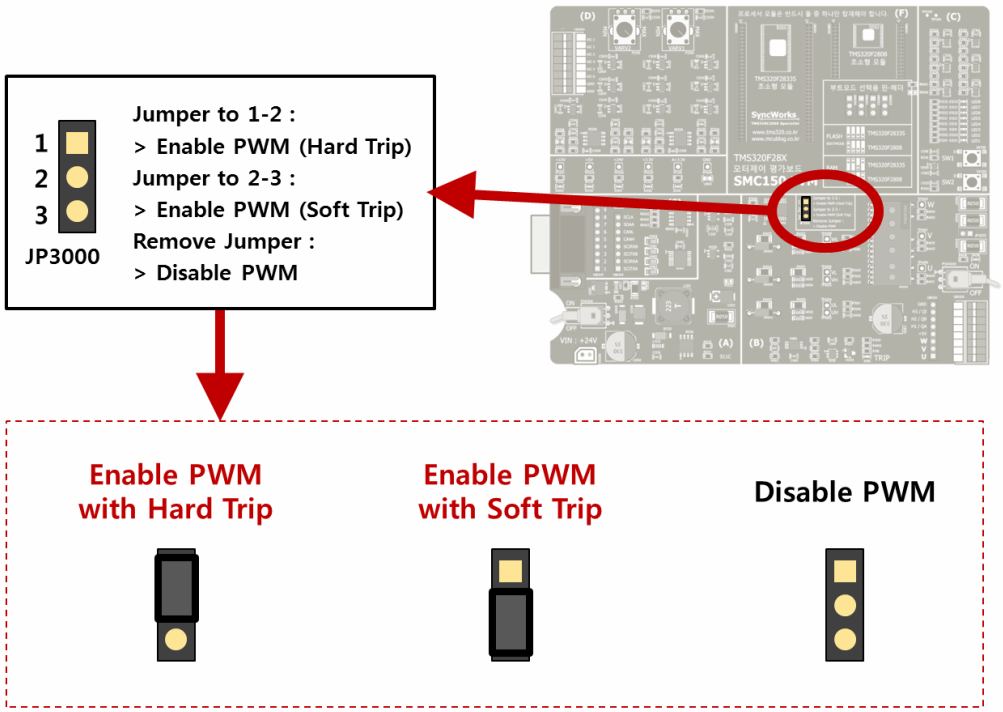
즉, 모터 제어 중 DC-Link에 흐르는 전류가 설정된 수치를 초과하면, 과-전류 Trip 회로에 의해 발생된 Trip 펄스가 TMS320F28X 프로세서의 PWM 출력을 모터 드라이버 IC에 전달하는 Voltage Translator를 Disable 하도록 설정할 수 있습니다. (Hard-Trip)

만약, 이 Trip 펄스가 Voltage Translator(SN74LVC4245A)의 /OE 핀에 직접 전달되지 않도록 설정하면, TMS320F28X 프로세서가 Trip 펄스를 감지하고, PWM 핀의 상태를 가장 안정한 상태로 고정하거나 범용 입출력 포트(GPIO) 조작을 통해 Voltage Translator를 Disable 하도록 할 수 있습니다. (Soft Trip)

이런 모든 설정은 JP3000 점퍼 핀-헤더(2.54mm 1x3)의 조작에 따라 이루어지며, 제품과 함께 제공되는 2P 점퍼로, PWM 출력을 Disable / Enable with Hard Trip / Enable with Soft Trip의 총 3가지로 제어할 수 있습니다.

예제 실행을 위해, 그림 1-5와 표 1-2를 참조하여, PWM 출력이 모터 드라이버 IC(Smart Power Module)에 전달될 수 있도록 해주십시오. Hard Trip 동작과 함께 PWM 출력을 Enable 할 경우, SMC150 개발보드에 설계된 과-전류 Trip 회로 출력에 의해, 자동으로 전류제한 동작이 이루어지며, Soft Trip을 선택하면, 예제에 의해 소프트웨어적으로 전류제한 동작이 이루어집니다.

JP3000 핀-헤더를 통해, PWM 출력이 Enable 되지 않은 경우, 예제를 실행해도 모터가 돌아가지 않습니다.



[그림 1-5] – JP3000 점퍼 핀-헤더의 위치 및 설정 예

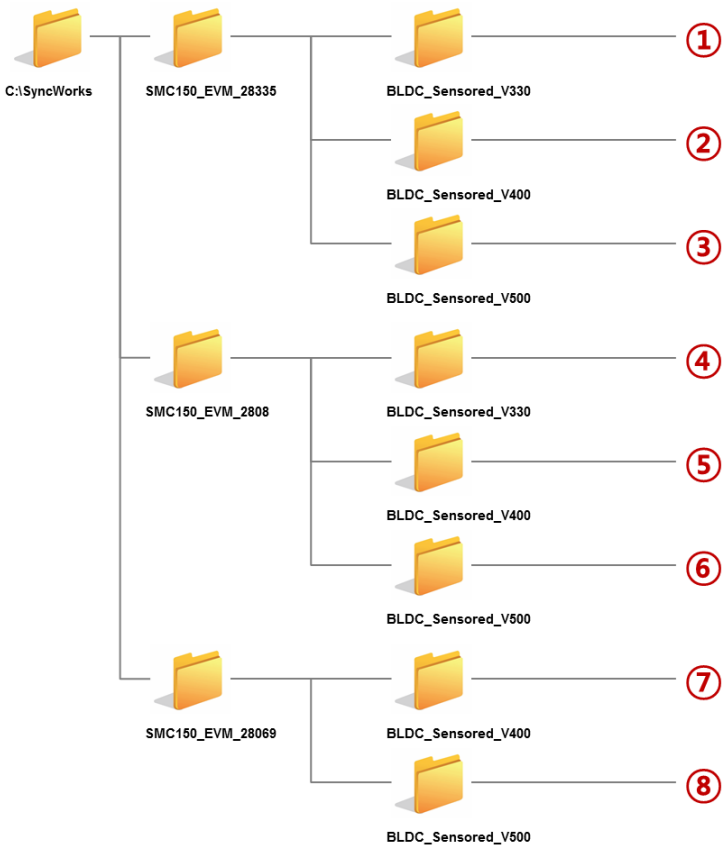
JP3000 핀-헤더	TMS320F28335	TMS320F2808	TMS320F28069 (어댑터를 통해 장착)
1 번 핀	과-전류 Trip 회로의 출력 펄스		
2 번 핀	Voltage Translator(SN74LVC4245A, U3000)의 /OE 핀		
3 번 핀	GPIO14/-TZ3/ -XHOLD/SCITXDB/MCLKXB	GPIO6/EPWM4A/ EPWMSYNCI/EPWMSYNCO	GPIO42/EPWM8A/ -TZ1/COMP1OUT

[표 1-2] – JP3000 점퍼 핀-헤더의 정의

2 예제 실습

SMC150 개발보드의 활용예제는, TMS320F28X 프로세서를 이용하여, 3상 BLDCM이나 PMSM의 토크, 속도, 위치 등을 제어해볼 수 있도록 구성되어 있으며, Texas Instruments(TI)사의 Digital Motor Control Library를 기초로, SMC150 개발보드에 적합하도록 일부 수정한 것 입니다. BLDCM의 경우 Hall-Effect Sensor가 주는 정보를 토대로 모터의 속력을 알아내고, 여기에 PI제어기를 붙여서, 사용자가 설정한 속력을 추종하는 예제가 제공되며, PMSM은 2,000C/T의 고-분해능 엔코더와 벡터 제어 기법(FOC)을 통해, 모터의 토크, 속도, 위치 등을 제어(PI)해볼 수 있도록 하는 예제가 제공되고 있습니다. 추후에도 사용자들의 요구에 맞춰, 다양한 활용예제들이 추가될 예정입니다. - TI의 최신 예제 및 라이브러리는 TI 웹 페이지가 아닌 ControlSUITE(클릭) 통해서 받으실 수 있습니다.

2.1 예제 디렉터리 색인



[그림 2-1] – BLDCM Sensored Control 예제 디렉터리 색인

그림 2-1의 1번과 2번, 3번 경로는 TMS320F28335 초소형 모듈 사용자를 위한 예제 프로젝트가 담겨있으며, BLDC_Sensored_V330 경로에는 CCS V3.3용, BLDC_Sensored_V400 경로에는 CCS V4.2.X용, BLDC_Sensored_V500 경로에는 CCS V5.2.X용 예제가 준비되어 있습니다.

4번과 5번, 6번 경로에는 TMS320F2808 초소형 모듈 사용자를 위한 예제 프로젝트가 담겨 있으며, 앞의 경우와 마찬가지로 XXX_V330은 CCS V3.3용 예제 프로젝트이고, XXX_V400은 CCS V4.2.X용 예제 프로젝트, XXX_V500은 CCS V5.2.X용 예제 프로젝트 입니다.

7번과 8번 경로에는 전용 어댑터를 통해 SMC150 개발보드에 장착되는 TMS320F28069 보급형 모듈 사용자용 예제 프로젝트가 담겨 있습니다. CCS 버전에 따른 경로 구분은 앞서의 경우와 동일합니다. 단, CCS V3.3의 경우, TMS320F28069 프로세서를 정식으로 지원하지 않기에 CCS V3.3용 예제는 제외되었습니다. ("C:\SyncWorks\TIDCS\..." 이하의 폴더에는 예제 실행에 필요한 TI 제공 칩-지원 헤더 및 소스가 위치하며, 그림 2-1에서는 생략되어 있습니다. – 예제 실행을 위해 반드시 설치해야 합니다.)

2.2 예제 구성

SMC150 개발보드용 BLDCM Sensored Control 예제는 총 2단계의 Build Level로 구분되어 있으며, 각 단계는 아래와 같습니다. Build Level 설정에 따라, 그에 적합한 코드가 Compile 되도록 구성되어 있으며, Build Level 설정은 예제 프로그램의 헤더(BLDC_Sensored.h)파일에 정의되어 있습니다.

- **LEVEL 1**

  Hall-Effect Sensor가 주는 정류시점에 따라 스위칭 패턴을 전환하여, 모터를 회전시키며, 일정한 Duty-ratio의 PWM 출력을 통해 Open-loop로 구동합니다.

- **LEVEL 2**

  Hall-Effect Sensor와 S/W 모듈을 통해, 현재 모터의 속력을 알아내고, PI 제어를 통해 사용자의 속력지령을 추종하도록 한 Closed-loop 구동단계 입니다. (별도의 전류제어 Loop는 구성되어 있지 않으며, 속력 정보를 토대로만 제어합니다.)

2.3 실습 따라 하기 (Build Level 1)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, BLDC_Sensored.h 파일에 정의된 Build Level이 "LEVEL1" 인지 확인합니다. (다르다면 LEVEL1로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel1.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel1.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 1은 일정한 Duty-ratio의 PWM 출력을 통해, 모터를 Open-loop로 구동하면서, Hall-Effect Sensor의 정보를 정확히 취득하고 있는지, PWM Duty-ratio 변화에 따라 모터의 속력이 달라지는지 확인하는 단계 입니다.

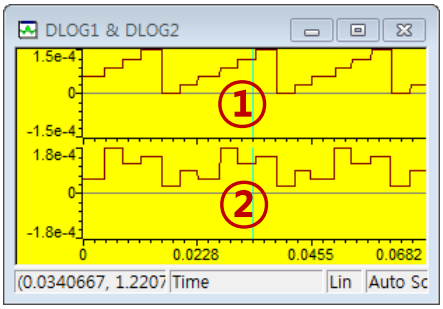
- **조작 대상 변수 → DFuncDesired (Q15 Format)**

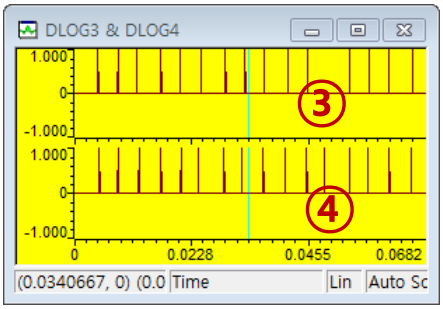
  DFuncDesired 변수를 0x0000 ~ 0x7FFF (Q15 Format으로 0.0 ~ +1.0) 사이에서 조작하면, 비율에 따라 출력되는 PWM의 Duty-ratio가 변화하고, 그에 따라 모터가 가속하거나 감속하게 됩니다. DFuncDesired 변수의 변화는 PWM Duty-ratio에 바로 반영되지 않고, Ramping 동작을 통해 점진적으로 적용되도록 구성되어 있습니다. (Q15 Format의 변환 예 - 만약, 30%에 해당되는 PWM Duty-ratio를 출력하고자 한다면, DFuncDesired 변수에는 $0.3 \times 2^{15} = 9{,}830$ [0x2666] 를 넣어줘야 합니다.)

- **주요 관찰 대상 변수 → speed1.SpeedRpm, hall1 구조체의 내부변수들**

  speed1.SpeedRpm 변수는 현재 모터의 속력을 RPM 단위로 표현합니다. PWM Duty-ratio 변화에 따라 달라지는 모터의 속력정보를 확인하고, hall1 구조체를 Watch Window에 등록하여, Hall-Effect Sensor로부터 정류시점을 파악하는 S/W 모듈의 동작에 대해 살펴보십시오.

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 1 에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 mod1.Counter, hall1.HallGpioAccepted, mod1.TrigInput, hall1.CmtnTrigHall 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.4 실습 따라 하기 (Build Level 2)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, BLDC_Sensored.h 파일에 정의된 Build Level이 "LEVEL2" 인지 확인합니다. (다르다면 LEVEL2로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel2.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel2.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

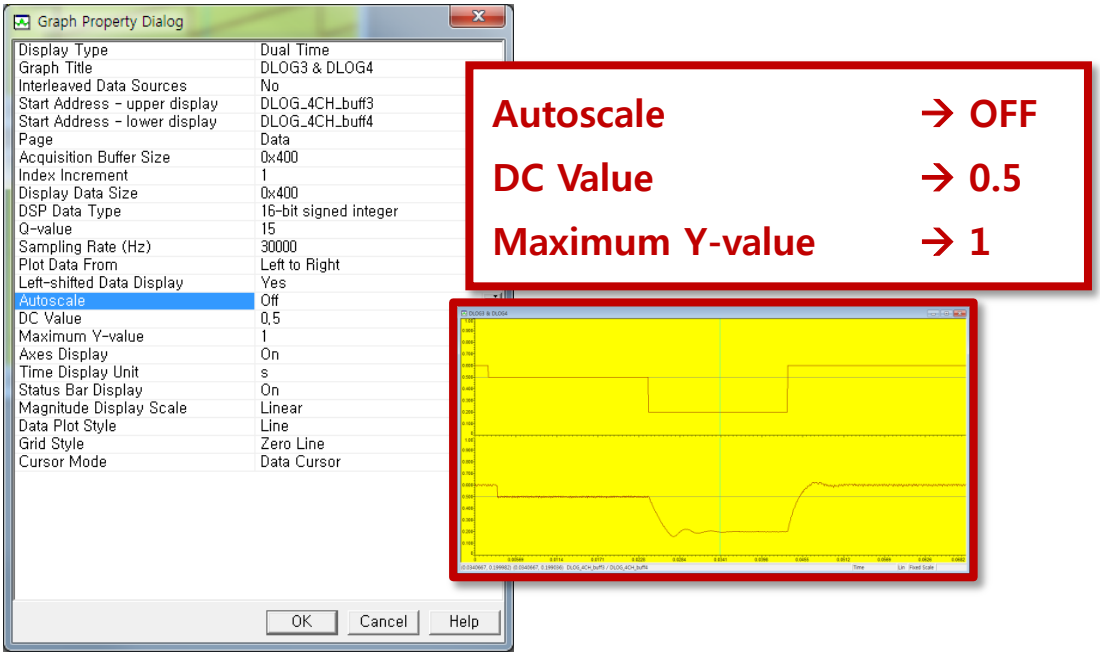
⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 2는 Hall-Effect Sensor로부터의 정보와 S/W 모듈(speed1)을 통해 현재 모터의 속력정보를 알아내고, 이를 기반으로 한 PI 제어 Loop를 통해, 모터의 속력이 사용자의 지령에 추종하도록 하는 단계 입니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해봅니다.

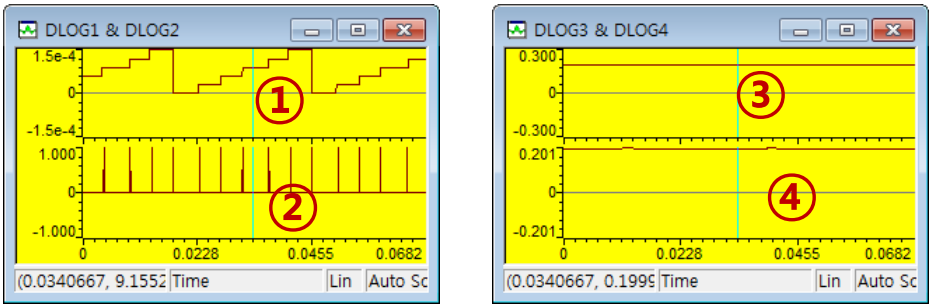
- **조작 대상 변수 → SpeedRef, SpeedKp, SpeedKi**

  SpeedRef 변수를 0.0 ~ +0.5 사이에서 변경하면, 속력제어용 PI 제어모듈의 지령 값으로 반영되어, 현재 속력정보와의 오차를 토대로 산출된 PI 제어기 출력 값 비율에 따라 모터 드라이버에 전달되는 PWM의 Duty-Ratio가 달라지게 됩니다. 이때, 모터의 회전자 측면에 손가락을 대고 살짝 힘을 가하면, PI 제어기에 의해 PWM Duty-ratio가 커지면서 모터가 사용자의 속력지령을 추종하는 모습을 확인할 수 있습니다. SpeedKp 변수는 속력제어용 PI 제어기의 Proportional Gain을 의미하며, SpeedKi는 Integral Gain을 나타냅니다. 이 값들을 다르게 설정한 후, SpeedRef 변수를 변경해보면, 달라진 Gain에 따라 지령속도를 추종하는 형태가 달라짐을 확인할 수 있습니다. (Watch Window에 등록된 dlog.prescalar 변수의 값을 "500"으로 변경하고, "DLOG3 & DLOG4" Graph Window를 Auto Scale이 아닌 Fixed Scale로 설정하시면, 추종과정의 속력변화 History를 보다 확실히 확인하실 수 있습니다.)



- **주요 관찰 대상 변수 → speed1.SpeedRpm, pid1_spd 구조체의 내부변수들**
  speed1.SpeedRpm 변수는 현재 모터의 속력을 RPM 단위로 표현합니다. SpeedRef 값 변화에 따라 달라지는 모터의 속력정보를 확인하고, pid1_spd 구조체로부터 32-Bit Digital PI 제어 동작에 대해 살펴보십시오.

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 2 에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 pwm1.CmtnPointer, hall1.CmtnTrigHall, pid1_spd.Ref, pid1_spd.Fdb 변수의 이력을 의미합니다.



⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

3 실습 끝내기 절차

① **Watch Window에 등록된 "EnableFlag" 변수를 0으로 변경합니다.** (변경 후 모터가 정지하는 것을 확인합니다.)

② **SMC150 개발보드의 DC-Link 전원 제어용 스위치(PSW4000)를 "OFF" 위치로 변경하여, 모터 드라이버 IC에 공급되는 전원을 차단합니다.**

③ Code Composer Studio에서 Halt(or Suspend) 명령으로 프로그램 실행을 멈추고, Real-time Mode도 해제합니다.

④ Code Composer Studio를 종료하고, SMC150 개발보드에 공급된 전원을 차단합니다.

⑤ JTAG 에뮬레이터와 프로세서 모듈을 분리합니다.